SFA Mobile & DMS

Manager walkthrough
Process, business logic and code architecture

Prepared from your local project handbook. Covers the employee app, distributor portal, backend, Salesforce classes, triggers and their relationships.

Start with this document for your demonstration. The complete companion PDF includes every numbered code explanation from the handbook: 1048 files and 34,517 source lines.

This is documentation of the 26 October 2026 snapshot, not a new deployment or a guarantee that every runtime path is tested. The verification chapter distinguishes checks that passed from remaining limitations.

Internal project documentation. Do not publish source/configuration publicly.

Reading map

1. Start here: what this project actually is

2. Folder map and reading order

3. SFA: authentication, day planning and outlets

4. SFA: GPS, visit actions, order saving and checkout

5. DMS: login, session and refresh

6. DMS: primary order and scheme algorithm

7. DMS: receipt → inventory, and why approval is not stock

8. Secondary orders, returns, tickets and allocation

9. Reports, offline behavior and deployment boundaries

10. Salesforce standards applied and remaining review work

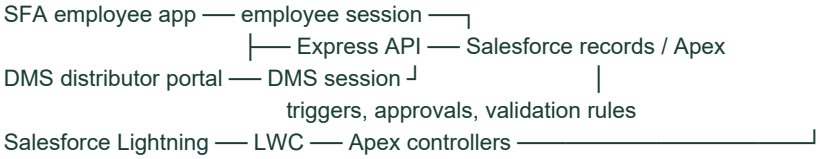
11. Manager questions: short answers to practise

12. Verification and change boundaries

Use the PDF bookmarks panel to jump directly to a topic. For a manager meeting, explain the process first; open the complete reference only when asked about a particular file or statement.

Start here: what this project actually is

This is one Salesforce data model used by two external clients: the **SFA employee mobile app** and the **DMS distributor portal/mobile client**. They share a Node/Express backend codebase. The Lightning components inside Salesforce are management screens; they are not the React Native mobile application.



Manager introduction: “The employee app records field activity. Salesforce is the central record system. The distributor portal reads the distributor’s related transactions and sends validated commands back through the backend. Primary transactions are case-based; secondary orders are usually piece-based. Approval, receipt and inventory posting are different steps.”

Use the file explorer for a numbered explanation beside every included source line. The line notes explain syntax and local operations; these chapters explain the business flow. They are not a claim that every possible runtime path was tested. Source hashes and the coverage manifest identify the exact local snapshot.

Included: first-party Apex, triggers, LWC, backend, both client source trees, Salesforce XML metadata, tests, configuration and operational scripts. Excluded: dependencies, compiled bundles, APKs, images/binaries, .env credentials, caches, diagnostics, old backups, generated data exports, lockfiles and earlier documentation. Those are not additional application logic.

Folder map and reading order

Folder	Responsibility
force-app/main/default/classes	Apex controllers, trigger services, REST commands and unit tests.
force-app/main/default/triggers	Salesforce save-event dispatchers. Before events validate/change incoming values; after events handle relationships and totals.
force-app/main/default/lwc	Three Lightning bundles: beat outlet selection, monthly PJP planning and employee picture upload.
force-app/main/default/objects	Object/field definitions, formula fields, picklists, validation rules, list views, sharing reasons and relationships.
sales-rep-mobile/backend/src	HTTP endpoints, identity checks, Salesforce transport, documents, reports and mock/test dependencies.
sales-rep-mobile/mobile/src	Employee screens, reusable fields, local drafts, sessions and offline queues.
dms-mobile/src	DMS login, desktop portal shell, orders, returns, receipts, stock, reports and sync.
scripts and launcher files	Local operation, seed/backfill and deployment utilities. Reading these is safe; running seed/reset scripts changes data.

Read in this order: SFA App.tsx → LoginScreen → api/client → backend app.ts → SalesforceService; then StoreGateScreen → VisitScreen → checkout. For DMS: App.tsx → DmsApp → api.ts → dms/router.ts → transport.ts → DmsPortalApi. Finish with triggers, handlers, metadata and tests.

Naming trap: the older `Distributor__c` is used as an SFA sales-representative record. The actual DMS distributor is `DMS_Distributor__c`. `Retailer__c` is the outlet. Labels can differ from API names; do not rename API names just to make terminology look nicer.

SFA: authentication, day planning and outlets

1. App.tsx registers the navigation screens and starts queue flushing when connectivity returns. Login is the initial screen.

2. LoginScreen collects an email, requests a code and exchanges a valid code for an employee session. The HTTP client attaches its bearer token to protected requests.

3. backend app.ts exposes login before the employee-authentication middleware. The DMS router has its own authentication and is mounted separately. Do not treat a DMS token as an employee token.

4. findEmployee resolves an active employee. The backend supplies the actor identity; clients must not be allowed to select an arbitrary employee ID to impersonate.

5. Day and attendance endpoints record start/end activity. Beats/outlets are limited by allocations. A cached list is not a fresh Salesforce read.

6. BeatsScreen and StoresScreen navigate to StoreGateScreen with the selected outlet; StoreGate refreshes the outlet so old coordinates are not the only source.

PJP flow: pjpMonthlyPlanner calls SfaPhaseOnePjpController. currentEmployee finds the logged-in Salesforce user’s employee mapping. saveDraft validates dates and authorized beats, writes a draft PJP and its PJP_Selected_Beat junction records. submitPjp checks the reporting manager and prevents self-approval. decidePjp requires the configured approver and records decision audit fields.

After approval, PjpApproverTrigger invokes SfaPjpAllocationService. It collects all selected beats, falls back to the legacy single Beat lookup when needed, locks employees, finds active outlets and creates/reactivates employee-outlet allocations. The pair key prevents repeated creation in its lookup logic. Allocation failure is not silently ignored: it can roll back the approval transaction.

BeatOutletSelectorController loads active outlets from the beat’s geography, enforces a maximum of ten selections, removes deselected assignments and assigns selected outlets. OutletGeographyTrigger copies geography and refreshes the affected old/new beat counts. PJP approval is not the same as a product-order approval.

SFA: GPS, visit actions, order saving and checkout

1. StoreGateScreen requests foreground GPS permission on the native phone and obtains the current position. In the **web preview**, the code deliberately uses the outlet's Salesforce coordinates. A successful web preview does not verify real mobile GPS.

2. The backend validates distance against the configured allowed radius. The distance helper uses the Haversine formula: convert degrees to radians, compute angular distance on a sphere and multiply by Earth's radius in metres.

3. VisitScreen starts/restores a visit and loads products. It stores form state separately from saved visit data. useMemo derives product filters and order totals; useRef retains stable request IDs across renders.

4. Brand, sub-brand and alias are derived from the selected product. Changing the selected product/filter clears dependent selection state to avoid showing another product's scheme.

5. Save action for an order first writes a local draft, then stores the draft on the visit in Salesforce. **A draft is not a submitted Sales Order.** On recovery, submitted order items take precedence over an old draft.

6. Other actions map to Product_Return, Competitor_Activity, SFA_Ticket, Stock_Check and Asset_Survey records. The backend verifies the visit belongs to the employee. Some endpoints have different retry guarantees; do not assume all writes are equally idempotent.

7. Complete Visit submits order details and checks out. The backend checks for a previously saved order and restores it rather than blindly creating another one. Salesforce composite requests request all-or-none handling and inspect individual subresponses, not just the outer HTTP 200.

8. Document services build PDF/Excel summaries. Sharing/download/email is separate from saving the order. A document-generation error does not itself prove the order was not saved.

Order header: Sales_Order__c. Product details: Order_Item__c. Parent Product__c stores the primary product for normal layouts; it is not a replacement for child lines. Product configuration minimums, selected schemes, GST and server-side product data affect the final saved amount.

Important existing limitation: the /api/store/revisit route currently returns a synthetic result; it does not persist a Salesforce revisit. The generic /api/orders/:orderId/items response also needs implementation review. These are documented as gaps, not working features.

DMS: login, session and refresh

1. DmsApp reads the stored session and the snapshot/queue keyed by distributor ID. The desktop PortalFrame and mobile navigation display the same underlying state.

2. Request-code validates/normalizes the email. The backend requires exactly one active distributor match. It uses cryptographic random six-digit codes, stores only a digest, expires them after ten minutes and limits verification attempts.

3. Resend is limited to prevent abuse. The previous challenge is replaced only after the mail send action is accepted. The generic response avoids revealing whether an email exists.

4. Verify-code compares digests with timingSafeEqual, removes the accepted challenge and rechecks the active distributor before issuing an eight-hour DMS JWT.

5. Protected DMS routes verify the token’s audience and recheck the distributor record. The body’s distributor identity is not trusted as the actor.

6. sync sends queued commands, retains failures and then fetches overview/report data. A 400 validation error requires correction; a timeout/unreachable backend leaves the saved request for retry.

7. Foreground refresh is periodic (20 seconds); reports are refreshed less frequently (about one minute unless requested). This is polling, not an instantaneous streaming subscription.

8. Logout invalidates the local session and uses generation checks so an older in-flight response cannot repopulate the next account’s screen. Pending work remains scoped to the original distributor.

Email delivery distinction: emailSimple success means Salesforce accepted the action. It does not prove Gmail delivered the message. This project previously encountered Gmail DKIM rejection; code formatting does not solve sender-domain authentication. Request-code, verify-code and email delivery are three separate checkpoints.

The OTP/rate-limit maps are process memory. Restarting the backend invalidates outstanding codes. Production multi-instance hosting needs a shared expiring store and a correctly authenticated mail sender.

DMS: primary order and scheme algorithm

1. PrimaryOrderForm loads the product catalogue, positive case prices and product-specific pieces-per-case conversions. Missing setup must not be invented by the UI.

2. The distributor selects whole cases and an applicable Primary scheme. Review is a confirmation step, not a saved order.

3. The distributor router sends distributorPrimaryOrder. The SFA employee route sends primaryOrder and derives both employee and assigned distributor from the authenticated employee session.

4. DmsTransport posts to `/services/apexrest/dms/portal/`. DmsPortalApi requires the DMS_Trusted_Backend custom permission and locks the active distributor before dispatch.

5. DmsEmployeeTransactions validates 1-50 distinct lines, positive whole quantities and stable request keys. A payload hash distinguishes a retry from reuse of the same key for different details.

6. It reads product prices/conversions from Salesforce, creates a Draft header, inserts lines and submits the configured Salesforce approval process. The assigned company Distributor Officer is the approver; the client does not choose an arbitrary approver.

7. DmsPrimaryWorkflow checks submission prerequisites, snapshots conversions/prices and makes submitted lines immutable. DmsOfficerSharing maintains read shares for the assigned officer.

Scheme qualification: active, Sales_Channel = Primary, within date limits, linked to the selected product directly or through Scheme_Product, no outlet-only restriction, and matching/unrestricted distributor geography. “No scheme” can be correct configuration behavior.

```
complete buy blocks = floor(ordered cases / minimum cases)
discount = round(cases × case price × percent / 100 + blocks × fixed discount, 2)
free cases = blocks × scheme free cases
gift quantity = blocks × configured gift quantity
```

The discount cannot exceed the paid line value. Free cases can reference another active product and must use that product’s case conversion. The server validates this again; displaying a scheme on the screen is not authorization to apply it.

Illustration only: 3 cases at ₹918 gives ₹2,754 before tax. At 5% GST and no discount, the total is ₹2,891.70. If a case contains 72 pieces, 3 cases means 216 pieces. These numbers are not universal product settings.

DMS: receipt → inventory, and why approval is not stock

Draft order → Submitted → Officer Approved
↓ actual delivery
Review goods receipt → confirm → Salesforce atomic command
↓
Receipt header + line + inventory change → Received → refreshed portal snapshot

DmsPortalApi.receiveGoods first checks whether the request key has already saved this exact receipt. If so it returns the original ID. If the same key describes different details, it rejects it. It then validates paid/free kind, whole cases, batch and expiry.

The order line must belong to this distributor and an Approved order. The server sums earlier received quantities, compares them with the paid/free entitlement and rejects over-receipt. It locks matching inventory for the distributor/product/batch and requires the existing batch expiry to match.

In one transaction it inserts a Draft receipt, inserts its line, adds `received cases × pieces per case` to stock, and marks the receipt Received. The outer savepoint rolls back partial changes on failure. Paid receipt case cost subtracts the allocated discount per case; free goods have zero purchase cost.

The app keeps the queued command until the API returns a valid saved ID/status. It reloads overview and stock; “pending” is not a saved receipt. A disabled receive button can indicate an exhausted approved quantity, a busy sync or invalid input. Read the actual condition beside the button in DmsApp before diagnosing it.

Directly creating arbitrary receipt records through the Salesforce UI is not automatically equivalent to calling this posting service. The inventory update shown here belongs to receiveGoods. Do not claim an inventory trigger handles every possible manual edit.

Secondary orders, returns, tickets and allocation

Secondary orders: employees save outlet Sales_Order records through SFA. DmsSecondaryRouting derives DMS_Distributor from the outlet and rejects an inconsistent/inactive supplier. Newly routed Sales_Order records receive Approved status, decision notes and timestamp automatically. Returns remain Pending. The portal no longer presents order approval/rejection controls, but older server decision methods remain in the codebase; hiding buttons is not endpoint retirement.

Auto-approval is not an invoice, physical shipment or automatic inventory deduction. Read invoice/stock-posting code and metadata separately before claiming dispatch is automated.

Primary returns: employee entry references a received goods-receipt line. The server locks receipts, sums non-rejected returns, checks remaining received cases and validates Damaged/Expired/Delayed Delivery/No Demand/Other. Other requires notes. It snapshots original receipt values and submits the company-officer return approval process. Primary and secondary return views are distinct in DMS.

Secondary returns: Product_Return records originate from outlet visits, normally in pieces. The outlet-to-distributor relationship routes them into DMS for the distributor's decision. Do not confuse an employee return with the distributor's own company-level return.

Tickets: DMS_Support_Ticket is distributor support, separate from outlet SFA_Ticket. DmsTicketRules stamps Open/Officer/Owner on creation. Open → In Progress or Rejected; In Progress → Closed or Rejected. Terminal decisions need resolution text. Only the assigned officer or DMS_Manage_Tickets permission can change the status.

Round-robin: DmsEmployeeAllocator selects active distributors ordered by Allocation_Order, locks the persistent allocation-state record, assigns each unassigned employee to the next distributor and wraps to the first. The cursor and employee changes commit together. Missing state is a configuration error; no active pool leaves allocation unchanged. Existing employee assignments are not randomly overwritten.

Discontinued: the lifecycle handler forces Active false when Discontinued is selected. Clearing the checkbox does not silently reactivate the login. History remains.

Reports, offline behavior and deployment boundaries

dashboard.ts collects distributor-scoped activity. reports.ts queries the full paginated record sets and calculates periods in Asia/Kolkata. Booked secondary sales are Submitted/Fulfilled orders excluding distributor rejection; this is not invoiced revenue or sales net of returns. Product breakdown uses line values while headers may include tax.

- Sales achievement = booked sales / configured target. Missing target is represented as unavailable, not a false 0% achievement. Web DMS hides sales-vs-target; native rendering retains the relevant report branch.

- Beat productivity distinguishes planned, visited and productive outlets. A productive outlet has a qualifying visit linked to an order.

- Outlet coverage counts active buyers against active assigned outlets. A visit alone is not necessarily a buyer.

- Stock metrics distinguish on-hand, available, low stock, expiry and no recent movement.

- Purchased-vs-sold uses posted receipt/invoice data; an approved purchase order alone is not purchased stock.

DMS session tokens use SecureStore on native and AsyncStorage on web. Snapshot and queue keys include distributor ID. SFA has a generic queue and a separate primary queue; the generic queue key is global and should receive an account-isolation/concurrency review. Offline support is limited to cached information and implemented queued actions. Authentication and uncached data still need a connection.

DMS local web is normally port 8082, DMS backend 4001 via its launcher, and SFA commonly 8081/4000. DMS api.ts has a web fallback of 4000, so the launcher/build environment must set the DMS URL correctly. On a phone, localhost means the phone, not the computer.

Start-DmsLocal builds/starts local services; Keep-DmsLocalRunning checks the ports and restarts missing processes; Open-DmsPortal launches the page; Install-DmsAutoStart installs per-user shortcuts. These do not make a laptop into permanent cloud hosting. Sleep/shutdown, expired Salesforce authorization, unavailable internet or email-provider rejection still affect the service.

Salesforce standards applied and remaining review work

This pass improves maintainability, not provenance: readable names, formatting and comments do not make AI-assisted code “human-authored.” No artificial mistakes or authorship claims were added.

- Consistent four-space Apex formatting and a pinned formatter with debug-check.

- Three thin triggers now delegate lifecycle, survey and target-key rules to named handlers with descriptive variables.

- Handler sharing choices explicitly preserve system-trigger behavior; the survey still checks the actual assigned officer.

- Added focused handler tests for a 200-row batch, one-query survey validation and historical-officer preservation.

- Repaired local missing semicolon/closing braces. Repaired legacy REST compilation issues without changing its JSON envelope: Exception suffix, void response serialization and an approval variable that shadowed the Approval type.

- Formatted first-party TypeScript/TSX and LWC without changing API names or intended logic. One test file was deliberately excluded because the formatter reported an unstable layout.

Not a complete security certification. Remaining review items:

- Employee__c still has two triggers (allocation and invitation). Salesforce does not promise their relative execution order. Consolidation requires a separately reviewed deployment; merely deleting a local trigger would not delete it from the org.

- with sharing governs record visibility, not CRUD/FLS. Older Aura controllers and system-mode paths need endpoint-by-endpoint permission review before production.

- The current portal API has a trusted-backend permission guard; the legacy DmsTransactionApi does not use that same guard. Do not expose the legacy endpoint to distributor users.

- Older PjpPlannerController.decidePlan does not contain the explicit assigned-approver check found in SfaPhaseOnePjpController.decidePjp. The retained LWC imports the latter; review access to the legacy controller separately.

- Invitation email results use partial-success sending without fully reporting all results; SfaSalesRepresentativeInvitationService escapes quotes, not full HTML. Localhost and pinned APK invitation URLs need environment configuration.

- Generic SFA queue dependency/concurrency behavior, broad CORS, remaining any types and large controller files merit focused follow-up work with tests.

- Deployment tests, object permissions, integration-user least privilege, sender authentication and device testing are separate checks. See the verification section for what actually ran.

References: Salesforce bulk trigger guidance; Salesforce Apex security; Apex formatter and debug-check. The local standards document is also retained.

Manager questions: short answers to practise

Why use a backend instead of putting Salesforce credentials in the APK?

The backend holds the integration credentials and applies employee/distributor scope. The phone receives an application session, not the org credential.

Why are there primary and secondary orders?

Primary is company-to-distributor in cases with company-officer approval. Secondary is distributor-to-outlet, generally in pieces, captured by employees and automatically approved under the current routing rule.

Why use Map and Set in Apex?

Set collects unique IDs; Map looks up related records without querying for each row. The handler queries the batch once and processes it in memory.

Why FOR UPDATE?

It locks records during decisions that must be serialized, such as allocation cursors, receipt quantities and inventory posting. It prevents two concurrent requests using the same old balance.

What is idempotency?

Retrying the same stable request returns the saved result rather than creating a duplicate. The payload must match; a new business operation needs a new key.

Why is a record visible in Salesforce but not in DMS?

Check the signed-in distributor, lookup relationships, status/query filters, permissions and last successful snapshot refresh. Do not immediately create another record.

Why not add stock when the officer approves?

Approval authorizes the purchase. Stock represents goods actually received; receiveGoods posts the receipt and pieces together.

Why does a scheme not appear?

Check Primary channel, active flag, dates, product linkage, geography, outlet-only restriction and case minimum. A secondary scheme is not automatically a valid primary scheme.

Does HTTP 200 mean everything saved?

No. Inspect the success envelope and Salesforce composite subresponses. The transport rejects invalid IDs/statuses and returned errors.

What is before versus after trigger?

Before changes the incoming record or adds validation errors without separate DML on it. After has saved IDs for related records, shares and totals.

Does with sharing enforce field security?

No. CRUD/FLS needs separate enforcement or a deliberate, documented trusted-system design.

Can I promise zero errors or always-online localhost?

No. Explain tested paths and dependencies. Local services need the computer; production needs suitable hosting, persistent state, monitoring, authenticated email and permissions.

Demo order: show distributor/employee/outlet relationships in Salesforce → sign into the correct client → show fresh catalogue and stock → place/review a primary order → show Salesforce header and product lines → officer decision → receive an approved line → show saved receipt and refreshed stock → show SFA secondary activity in DMS → show a return/ticket → explain sync status. Do not approve or create real transactions only to illustrate the guide.

Verification and change boundaries

Check	Observed result on 26 October 2026
Salesforce check-only deployment	0AfjV000004xhxuSAA · Succeeded · 50 tests completed, 0 failures. Classes and triggers validated against sfaNewOrg. Check-only: no deployed code change.
Backend	39 tests passed; 8 skipped; TypeScript build passed. Local fixtures/mocks, not end-to-end email delivery.
SFA mobile	26 tests passed; TypeScript check passed.
DMS	UI, desktop/native navigation, logout, account-isolation and rejected-receipt correction scripts passed; TypeScript check passed. No real network calls in UI scripts.
Formatter	Apex debug-check passed before write. TS/LWC debug-check passed except visitRecovery.test.ts (formatter layout instability); that test was left unchanged.

Backups: backups/code-readability-20261026. No actual metadata deployment, record creation, approval, email sending, APK release or Git push was performed for this explanation task. Newly added handler classes/trigger dispatch changes must be deployed together after review.

Runtime code changes were limited to extracting three existing trigger rules and repairing the identified Apex compilation errors. Other changes are formatting. No wholesale claim of production/security compliance is made.